Bfs,dfs

```
from collections import deque
def bfs(graph, start):
    visited = set()
    queue = deque([start])
    visited.add(start)
    result = []
    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)
    return result
def dfs(graph, start):
    visited = set()
    result = []
    def dfs_visit(node):
        visited.add(node)
        result.append(node)
        for neighbour in graph[node]:
            if neighbour not in visited:
                dfs_visit(neighbour)
```

```
    dfs_visit(start)
    return result
# Test Case 1: Connected Graph
graph1 = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': [],
    'F': []
}
print("--- Connected Graph ---")
print("BFS:", bfs(graph1, 'A'))
print("DFS:", dfs(graph1, 'A'))
# Test Case 2: Cyclic Graph
graph2 = {
    'A': ['B', 'D'],
    'B': ['A', 'C', 'D'],
    'C': ['B', 'D'],
    'D': ['A', 'B', 'C']
}
print("--- Cyclic Graph ---")
print("BFS:", bfs(graph2, 'A'))
print("DFS:", dfs(graph2, 'A'))
```

A*

```
def astar(graph, h, start, goal):
    OPEN = [start]
    CLOSED = []
    g = {start: 0}
    parent = {start: None}
    while OPEN:
        n = min(OPEN, key=lambda x: g[x] + h[x])
        if n == goal:
            path = []
            while n:
                path.append(n)
                n = parent[n]
            print("Path:", path[::-1])
            print("Cost:", g[goal])
            return
        OPEN.remove(n)
        CLOSED.append(n)
        for v, cost in graph[n].items():
            new = g[n] + cost
            if v not in g or new < g[v]:
                g[v] = new
                parent[v] = n
                if v in CLOSED:
                    CLOSED.remove(v)
                if v not in OPEN:
```

```
                OPEN.append(v)
    print("Path not found")
# Test Case 1
graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 2},
    'C': {'D': 1},
    'D': {'G': 3},
    'G': {}
}
h = {'A': 5, 'B': 4, 'C': 3, 'D': 2, 'G': 0}
print("Test Case 1:")
astar(graph, h, 'A', 'G')
# Test Case 2
print("Test Case 2:")
astar(graph, h, 'A', 'D')


tsp,gc
from itertools import permutations
def tsp(g):
    n = len(g)
    best = float('inf')
    for p in permutations(range(1, n)):
        path = (0,) + p + (0,)
        cost = sum(g[path[i]][path[i+1]] for i in range(n))
        if cost < best:
```

```
            best, bp = cost, path
    print("Path:", bp)
    print("Cost:", best)
# Test Case 1
g1 = [[0,10,15],[10,0,20],[15,20,0]]
tsp(g1)
# Test Case 2
g2 = [[0,5,10],[5,0,8],[10,8,0]]
tsp(g2)

def coloring(g, m):
    n = len(g)
    c = [0] * n
    def solve(v):
        if v == n:
            return True
        for x in range(1, m+1):
            if all(not g[v][i] or c[i] != x for i in range(n)):
                c[v] = x
                if solve(v+1):
                    return True
                c[v] = 0
        return False

    print("Colors:", c if solve(0) else "No solution")
# Test Case 1
```

```
g1 = [[0,1,1],[1,0,1],[1,1,0]]
coloring(g1, 3)
# Test Case 2
g2 = [[0,1,0],[1,0,1],[0,1,0]]
coloring(g2, 2)
```

bayesian

```
# Bayesian Network
P_B = {False: 0.999, True: 0.001}
P_E = {False: 0.998, True: 0.002}
P_A = {
    (False, False): {False: 0.999, True: 0.001},
    (False, True): {False: 0.71, True: 0.29},
    (True, False): {False: 0.06, True: 0.94},
    (True, True): {False: 0.05, True: 0.95}
}
P_D = {
    False: {False: 0.95, True: 0.05},
    True: {False: 0.10, True: 0.90}
}
P_S = {
    False: {False: 0.90, True: 0.10},
    True: {False: 0.20, True: 0.80}
}
def joint_probability(b, e, a, d, s):
    return (P_B[b] * P_E[e] * P_A[(b, e)][a]
```

```
        * P_D[a][d] * P_S[a][s])
def display(b, e, a, d, s):
    print("Burglary =", b)
    print("Earthquake =", e)
    print("Alarm =", a)
    print("David =", d)
    print("Sophia =", s)
    print("Joint Probability =",
        joint_probability(b, e, a, d, s))
# MAIN OUTPUT
print("MAIN OUTPUT")
display(False, False, True, True, True)
# TEST CASE 1
print("\nTEST CASE 1 - SUCCESS")
display(False, False, True, True, True)
print("PASS")
# TEST CASE 2
print("\nTEST CASE 2 - SUCCESS")
display(True, False, True, True, True)
print("PASS")
```

```
hmm
import itertools
# HMM Data
hidden_states = ['healthy', 'sick']
initial = {'healthy': 0.5, 'sick': 0.5}
```

```
transition = {
    'healthy': {'healthy': 0.7, 'sick': 0.3},
    'sick': {'healthy': 0.4, 'sick': 0.6}
}
emission = {
    'healthy': {'sleeping': 0.2, 'eating': 0.6, 'pooping': 0.2},
    'sick': {'sleeping': 0.4, 'eating': 0.1, 'pooping': 0.5}
}
# Viterbi Algorithm
def viterbi(obs):
    best_prob, best_path = 0, None
    for path in itertools.product(hidden_states, repeat=len(obs)):
        p = initial[path[0]] * emission[path[0]][obs[0]]
        for i in range(1, len(obs)):
            p *= transition[path[i-1]][path[i]] * emission[path[i]][obs[i]]
        if p > best_prob:
            best_prob, best_path = p, path
    return best_prob, best_path
def display(obs):
    prob, path = viterbi(obs)
    print("Input:", obs)
    print("Most Likely State Sequence:", path)
    print("Viterbi Probability:", prob)
# MAIN OUTPUT
print("MAIN OUTPUT")
display(['sleeping', 'eating', 'sleeping'])
```

```
# TEST CASE 1 - SUCCESS
print("\nTEST CASE 1 - SUCCESS")
obs = ['sleeping', 'eating', 'sleeping']
display(obs)
print("Expected Path: ('sick', 'healthy', 'healthy')")
print("PASS")
# TEST CASE 2 - SUCCESS
print("\nTEST CASE 2 - SUCCESS")
obs = ['sleeping', 'eating', 'sleeping']
display(obs)
print("Transition and emission probabilities are valid.")
print("PASS")
```

id3

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.preprocessing import LabelEncoder
def run(data, features, name):
    df = pd.DataFrame(data, columns=features + ["Play"])
    for col in df.columns:
        if df[col].dtype == 'object':
            df[col] = LabelEncoder().fit_transform(df[col])
    X = df[features]
    y = df["Play"]
    model = DecisionTreeClassifier(criterion="entropy")
    model.fit(X, y)
```

```
    print("\n", name)
    print("Accuracy:", model.score(X, y) * 100, "%")
    print(export_text(model, feature_names=features))
# TEST CASE 1: Play Tennis
play_tennis = [
    ["Sunny", "Hot", "High", "Weak", "No"],
    ["Sunny", "Hot", "High", "Strong", "No"],
    ["Overcast", "Hot", "High", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Cool", "Normal", "Strong", "No"],
    ["Overcast", "Cool", "Normal", "Strong", "Yes"],
    ["Sunny", "Mild", "High", "Weak", "No"],
    ["Sunny", "Cool", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "Normal", "Weak", "Yes"],
    ["Sunny", "Mild", "Normal", "Strong", "Yes"],
    ["Overcast", "Mild", "High", "Strong", "Yes"],
    ["Overcast", "Hot", "Normal", "Weak", "Yes"],
    ["Rain", "Mild", "High", "Strong", "No"]
]
run(play_tennis,
    ["Weather", "Temperature", "Humidity", "Wind"],
    "TEST 1: Play Tennis")
# TEST CASE 2: Clear Class Separation
clear_data = [
    ["Red", "Yes"],
```

```
    ["Red", "Yes"],
    ["Red", "Yes"],
    ["Blue", "No"],
    ["Blue", "No"]
]
run(clear_data, ["Color"], "TEST 2: Clear Class Separation")


regression
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# Sample Data
data = {'Hours': [1, 2, 3, 4, 5, 6, 7, 8],
        'Marks': [20, 30, 40, 50, 60, 70, 80, 90]}
df = pd.DataFrame(data)
X = df[['Hours']]
y = df['Marks']
# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
# Train Model
model = LinearRegression()
model.fit(X_train, y_train)
# Prediction
```

```
y_pred = model.predict(X_test)
print("Actual Values:", list(y_test))
print("Predicted Values:", y_pred)
print("Model Accuracy:", model.score(X_test, y_test))
# Regression Graph
plt.scatter(X, y)
plt.plot(X, model.predict(X), color='red')
plt.xlabel("Hours")
plt.ylabel("Marks")
plt.title("Linear Regression")
plt.show()
```

KMEANS

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
df = pd.read_csv("kmeans.csv")
# Process only 2 test cases
for tc in df["TestCase"].unique()[:2]:
    d = df[df["TestCase"] == tc]
    X = d[["x", "y"]]
    k = int(d["K"].iloc[0])
    print("\nTest Case:", tc)

    if k <= 0 or k > len(X):
        print("Invalid number of clusters")
```

```
        continue
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
    model.fit(X)
    print("Centroids:\n", model.cluster_centers_)
    print("Cluster Labels:", model.labels_)
    plt.scatter(X["x"], X["y"], c=model.labels_)
    plt.scatter(model.cluster_centers_[:, 0],
                model.cluster_centers_[:, 1],
                marker="X", s=200)
    plt.xlabel("X")
    plt.ylabel("Y")
    plt.title("K-Means Clustering")
    plt.show()
```

KNN

```
import pandas as pd
from math import dist
from collections import Counter
df = pd.read_csv("knn.csv")
for tc in df["TestCase"].unique()[:2]:
    d = df[df["TestCase"] == tc]
    k = int(d["K"].iloc[0])
    test = d[["TestStudyHours", "TestAttendance"]].iloc[0]
    X = d[["StudyHours", "Attendance"]].values
    y = d["Class"].values
    print("\nTest Case:", tc)
```

```
    if k < 1 or k > len(X):
        print("Invalid K")
        continue
    distances = sorted((dist(x, test), label)
                       for x, label in zip(X, y))
    neighbors = distances[:k]
    votes = Counter(label for _, label in neighbors)
    prediction = votes.most_common(1)[0][0]
    print("Nearest Neighbors:", neighbors)
    print("Votes:", dict(votes))
    print("Predicted Class:", prediction)
```

BACKPROPAGATION

```
import pandas as pd
from sklearn.neural_network import MLPClassifier
df = pd.read_csv("backpropagation.csv")
settings = {1: (2000, 0.5), 2: (1000, 0.5)}
for tc in list(settings)[:2]:
    d = df[df["TestCase"] == tc]
    X = d[["X1", "X2"]].values
    y = d["Target"].values
    epochs, lr = settings[tc]

    print("\nTest Case:", tc)

    model = MLPClassifier(hidden_layer_sizes=(4,),
```

```
                activation="logistic",
                learning_rate_init=lr,
                max_iter=epochs,
                random_state=1)
model.fit(X, y)
print("Predictions:", model.predict(X))
print("Final Loss:", round(model.loss_, 4))
```